Plectis: What a Stranger Can Check

Bounded public evidence from a private system

Will Cook

July 2026

Abstract

Plectis is a public software repository (an organised, copyable project folder) built out of a larger private system that cannot be published. I treat it as a case study in a general problem: what may a stranger reasonably conclude from a curated set of small, runnable public fragments of work they cannot see? Each of the 88 public components (the repository's registered, separately testable parts) joins a claim to a fixed input, a command, stored output records, an explicit pass rule, and a stated limit; the count is an inventory, not a score. The public repository lets a reader test claims about these published files and their outputs. It cannot, by itself, establish the private system's history, the representativeness of the selection, or the correctness of success criteria the project supplies for itself. I developed the private system over about ten months by directing AI coding agents; that account is reported, not proved, and the argument does not depend on it. The paper examines one run from input to limit, sets out five distinctions that govern what runs can and cannot show, itemises the choices that remain in my hands and what those choices could quietly do, and states what stronger evidence would require, with a dated reproduction record in an appendix. One successful run supports one bounded conclusion; the paper is about which conclusions those are.

1 The problem

I have a larger private software system that I cannot responsibly publish: it contains personal records, credentials, live browser and account information, my working notes, and unfinished tools. I would still like some of its claims to be answerable to strangers, rather than resting entirely on my say-so. *Answerable* has a definite meaning here: starting from public material alone, a stranger can make something happen that bears on the claim, can inspect the rule by which the outcome is judged, and can aim any objection at a particular public object rather than at my account of a hidden one. Neither the difficulty nor that meaning is specific to me, and this paper examines what such answerability can amount to: what may a stranger reasonably conclude from a curated set of runnable public fragments of a system they cannot see?

Plectis is my working answer: a public software repository, stored at github.com/wcook04/plectis. A *repository* is an organised project folder, usually kept with its revision history; to *clone* a repository is to copy that folder and history to another computer. Everything this paper says a stranger can check is in that repository, and the paper's task is to describe the answer and measure how far it reaches.

I am 22 and study economics. Over about ten months I built the private system by directing AI coding agents: I set its objectives, rules, and review criteria, while the agents were the principal implementers, writing, running, testing, and revising the software under my direction. No other person worked on it. That account is testimony about origin, stated once here and not relied on afterwards: Plectis can expose the resulting public files and runs, but it cannot independently prove how the private work was produced. What the account does explain is why the publication problem arose, and why every published choice described below is my responsibility rather than the agents'. The artefact does not prove the story, and the evidence below would be exactly as strong, and exactly as limited, if every line had been written by hand.

The boundary of the evidence can be stated before any command is run:

Status	**What belongs in it**
Publicly checkable	The contents of a named public commit, its registered commands, the files those commands write, and the results of its public tests.
Reported here	The private system's history, the role of AI agents in producing it, and the private origin of published material.
Not established here	Whether the public selection represents the private whole, whether the mechanisms generalise beyond their fixed cases, whether self-supplied success criteria are independently correct or even aimed at the stated claims, and whether the private system works reliably at all.

Reading this paper requires no programming. Reproducing its runs requires a terminal (the text window in which commands are typed), Git (the standard program for copying and versioning repositories), the Python programming language (version 3.11 or later), the `pip` installer (the standard tool for fetching and installing Python software), and basic command-line use. So that the body of the paper can stay with the argument, the exact commands, environment details, and installation notes are collected in the appendix, which is a dated record of one verification pass at the commit named there.

2 The component contract

A *component* in Plectis is one registered, testable part of the project. Its *fixture* is a fixed sample input, and its *receipt* is a saved, machine-readable record of a reference run. A *validator* is the program that applies a component's pass and refusal rules. A *commit* is a named, saved version of the repository. These terms describe files and commands; they do not grade the importance of what any component does. (Inside the repository a component is called an *organ*, and the code package is named `microcosm_core`: the project was previously called Microcosm, and several file and command names keep the old name. The generated page `ORGANS.md`, named for the old term, gives a plain description and a first command for every component.)

At commit `57ffb7f5830c` the registry, the machine-readable list of components, contains 88 entries. Every entry is required to connect the same things: a claim stated in prose; a fixture; a command; the output files the command writes; a stored receipt from a reference run; an explicit rule for what counts as a pass and what must be refused; and a stated limit on what a pass establishes. Figure 1 lays the required parts out, together with the fact about them that the rest of this paper keeps returning to: every part is supplied by me, and every part is exposed to a stranger.

An assertion about private software can only be believed or doubted as testimony. A component converts the assertion into a procedure, and a procedure can fail in front of a stranger. The conversion does not make the claim true. What it changes is the form disagreement can take: a reader who doubts a component's claim can point at a particular input, a particular line of the validator, or a particular number in the output, instead of arguing with my description of software they cannot see. A component that fails is still answerable in this sense, and a

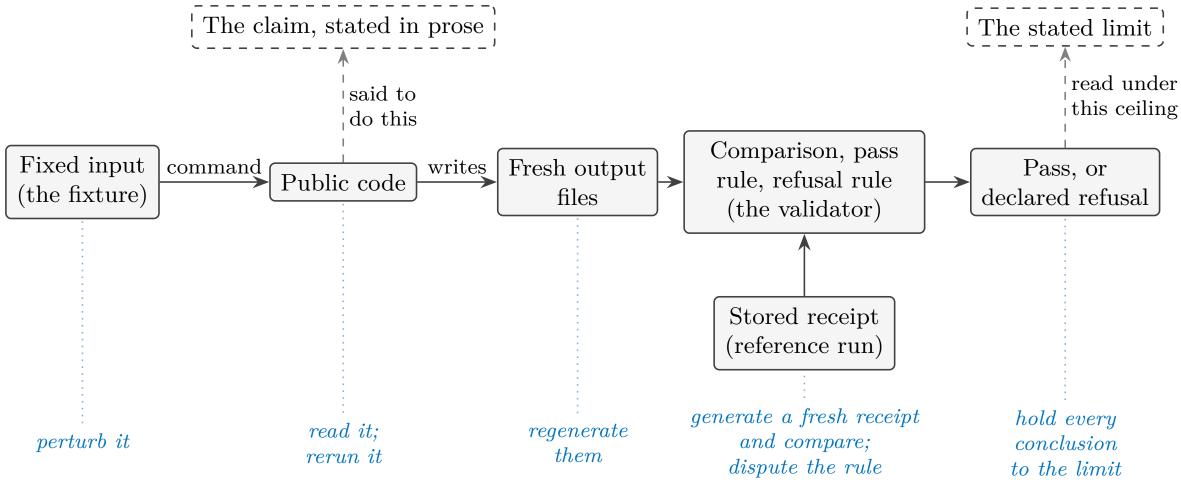

Figure 1: What every registered component must connect, at commit `57ffb7f5830c`. The top row is words; the middle rows are mechanism; a pass from the validator is offered as evidence for the claim and must be read under the limit. Every element, words and mechanism alike, is supplied by the project; dashed outlines mark prose rather than something that runs. The blue row names what a stranger can do to each part without asking anyone.

component can be answerable and trivial; answerability concerns the form a claim takes, not its truth and not its importance.

A component may also end in a declared refusal rather than an answer. The worked example below refuses an input format it does not support. Components that depend on an external tool, such as the Lean proof checker (a program that verifies mathematical proofs), must record a bounded refusal when the tool is absent, rather than reporting success. *Runnable* therefore means that the operation and its declared failure boundary are present in public form; it does not mean that every optional dependency exists on every machine. A refusal is informative only when it is declared in advance and distinguishable from a breakdown: a predeclared refusal marks the edge of what the component claims, while an unexpected crash is a defect. The two must not be read as the same kind of event. One honesty condition on refusals cannot be discharged from inside the repository at all: the public record shows each boundary as declared at the named commit, and it cannot show whether the boundary was drawn before or after an awkward case was first met. A boundary drawn afterwards turns a failure into a designed refusal in the telling. Freezing a version before an outside evaluation begins, as Section 7 proposes, is what would make that ordering checkable.

Little of the machinery is essential to the idea. What the contract needs is a claim in words; a determinate public case; a procedure a stranger can cause to run; an observable result; a rule connecting result to claim; a declared boundary of application; and a stated ceiling on inference. That the case here is a file of numbers, the procedure a command line, and the record a text document, is local convenience. The same contract could be satisfied in another field by a laboratory protocol with declared rejection conditions, or by a sealed evaluation service with published scoring rules. I make no claim about those settings; the point is only that the contract is a shape for exposing claims to strangers, and this repository is one implementation of it.

The property being built has familiar neighbours and is none of them. It is not reproducibility, though it uses it: a run can repeat perfectly with no stated claim attached, and a claim can be answerable while still awaiting a decent test. It is not transparency: the design begins from the fact that most of the system stays hidden, and offers a bounded point of contact instead of a view inside. And answering for the public form is not answering for the hidden whole: what a component exposes to challenge is the published fragment, and Section 4 is largely about how little any result against the fragment entitles anyone to conclude beyond it.

3 One run, examined

The component `batch8_audio_level_rms_port` computes a loudness level from short arrays of audio samples. I chose it for the worked example because it is ordinary. It needs no mathematics beyond a square root and no knowledge of the private system, and the claim it makes is small. Its value here is that every part of the evidence can be seen at once.

The reported origin is this. The private system contains a small macOS recording application, and one of that application's functions reduces a short stretch of audio samples held in memory to a level between zero and one. The public component re-expresses that calculation in Python. The component's code and its stored receipts name the private file it claims to re-express (`AudioLevelMonitor.swift`); a stranger cannot open that file, so the name is a report, not evidence. What the stranger can check is everything else.

The declared calculation is the standard one called root-mean-square (the `rms` in the component's name): square each sample, average the squares, take the square root, then multiply by eight and cap the result at one. Sixteen-bit integer samples are first divided by 32,767, the largest value a sixteen-bit integer can hold, to bring them onto the same scale. An empty input must produce zero. An unrecognised format name must be refused rather than guessed at.

The component's public command runs it against its fixture and writes fresh output outside the repository (the backslashes split one long command across lines):

```
PYTHONPATH=src python3 -m \
  microcosm_core.organs.batch8_audio_level_rms_port run \
  --input fixtures/first_wave/batch8_audio_level_rms_port/input \
  --out /tmp/plectis-audio-rms \
  --acceptance-out /tmp/plectis-audio-rms-acceptance.json
```

At commit `57ffb7f5830c`, the run produced:

Case	**Input**	**Expected**	**Observed**	**Result**
Decimal samples	0.0, 0.05, −0.05, 0.1	0.489897949	0.489897949	pass
Sixteen-bit integer samples	0, 1638, −1638, 3277	0.489892970	0.489892970	pass
Values above the cap	1.0, −1.0, 0.5	1.000000000	1.000000000	pass
Empty input	(none)	0	0	pass
Unknown format	`pcm24`	refusal	refusal	pass

Because the inputs and the rule are both printed above, the first row can be checked without the repository or a computer: the squares of the four samples average to 0.00375, whose square root is 0.061237…, and eight times that is 0.489897…. A reader with a calculator can recompute this expected value from the stated formula alone. For this one case the reader can act as their own *oracle*, the general name for whatever determines what an answer should have been, independently of the program under test.

Almost nothing else in the collection is like that, which is why the example matters. Elsewhere the project supplies the implementation, the input, the expected values, and the validator, all from the same hand. A test assembled that way can pass for four different reasons: the implementation and the expectation are both correct; the two share a mistake; the chosen cases happen to sit where the implementation behaves; or the validator checks a weaker property than the prose claim suggests. A matching result therefore establishes repeatability under that public test. Independent correctness needs an oracle that does not depend on me, as elementary arithmetic does for the first row, or fresh inputs whose expected results I never chose.

The example generalises into a question that can be asked of any test in any project: where does the expected value live, and who put it there? The positions differ in kind, not only in degree:

Where the oracle lives	**What a matching result then supports**
The project itself: implementation, cases, expectations, and rule from one hand.	Repeatability under the project's own test; the four readings above all stay open.
The reader's own derivation from stated premises, as in the worked example's decimal-samples row.	Bounded correctness of that one calculation on that one case.
An established external tool or reference.	Whatever the reference itself warrants, on the cases run, within the reference's own limits.
An evaluator who chooses inputs and expected results after a version is frozen.	Correctness on the evaluated cases, with my control over the answer removed.
The world: an outcome observed apart from any program.	A bearing on usefulness or empirical adequacy, where such an outcome exists at all.

Every component in the collection currently sits in the first position, except where a reader promotes a case to the second by recomputing it, as the worked example's decimal-samples row invites. The staged programme of Section 7 is, among other things, a plan for moving selected components down this table.

The refusal row records a designed boundary, not a failure: an input naming an unsupported encoding (`pcm24`, a format this component does not handle) must be rejected with an error, and is. The stored reference for the whole table is `batch8_audio_level_rms_port_validation_receipt.json` under `receipts/first_wave/batch8_audio_level_rms_port/`, so a fresh run is compared against a fixed record rather than against memory. A stored record by itself does not prove that the stated command produced it; its use is that anyone can generate a fresh one and compare the two.

The limit is part of the component. This run checks one public Python calculation over synthetic sample arrays (arrays composed for the test, not captured from a device). It does not run the original application, start an audio session, request microphone permission, or capture sound, and it cannot establish that the named private file is where the calculation came from. Nor does the table show that the task was difficult (it was not), that the component is useful elsewhere, or that the remaining entries resemble it. What it shows is that one assertion now has a form in which a stranger can rerun it, dispute its rule, perturb its input, or replace my expected value with a better-derived one.

4 Five distinctions

Interpreting the collection needs five distinctions. Each separates something a stranger can observe from something the observation may seem to imply, and each names the further evidence that would be needed to cross the gap. The limits attached to individual components are instances of these, and Figure 3 at the end of the section sets the five side by side.

Public execution versus private provenance

A reader can establish what a named public commit contains and what its commands do. No public run establishes where the material came from. Plectis keeps records at that boundary: copied files carry public import records naming their claimed sources, together with fingerprints (a *fingerprint* is a short value computed from a file's contents, chosen so that any change to the contents changes the value). Matching fingerprints show that a public file agrees with the project's public record of it. But I control both sides of that comparison, so agreement is evidence of internal consistency, not of origin. The same holds for the audio component's named source file: the name is an entry I wrote. Origin claims could only be strengthened by something a public repository cannot supply about itself, such as commitments published before the fact, or a trusted person permitted to examine both sides. Until then, provenance is reported.

Repeatability versus correctness

A run that reproduces the stored receipt shows that the public code, on the supplied input, in a sufficiently similar environment, produces the same result again. Correctness is a different property: it needs a standard for what the result should have been, and a standard is only worth something if it does not merely restate the implementation. The worked example makes the difference concrete. Its first row has an oracle independent of the project, namely elementary arithmetic; most components have only the project's own expected values and validator. Passing tests anywhere in Plectis should be read at that strength and no more.

A validator's rule versus the stated claim

A pass is a verdict from the validator, and a validator checks exactly the property written into it, which can be narrower than the claim as worded. A component whose prose speaks of handling a class of inputs may in fact verify that one output file exists and matches a stored shape; a rule can hold while the sentence it stands under goes untested. The two previous distinctions do not cover this gap: a result can be repeatable, and correct as far as the checked property goes, while the checked property is beside the stated point. Plectis makes the gap inspectable rather than closed. Every validator is public, so a reader can put the rule beside the prose and object where the rule is weaker, and the review in Section 8 makes exactly that comparison its central step. Since I wrote both the claims and the rules, agreement between them is my judgement recorded twice, and no count of passing components can substitute for reading one rule against one sentence.

Selected cases versus general behaviour

This distinction applies twice. Within a component, the fixture's cases are a few points in a large space of possible inputs, and behaviour elsewhere is untested. Across the collection, the published components are a selection from a private whole, and the selection was mine. I chose which mechanisms could be given public form, which inputs to freeze, which expected values to store, and which pass rules to enforce. Nothing in the public record shows how many candidates there were, what was excluded beyond the privacy rules, or whether the collection resembles

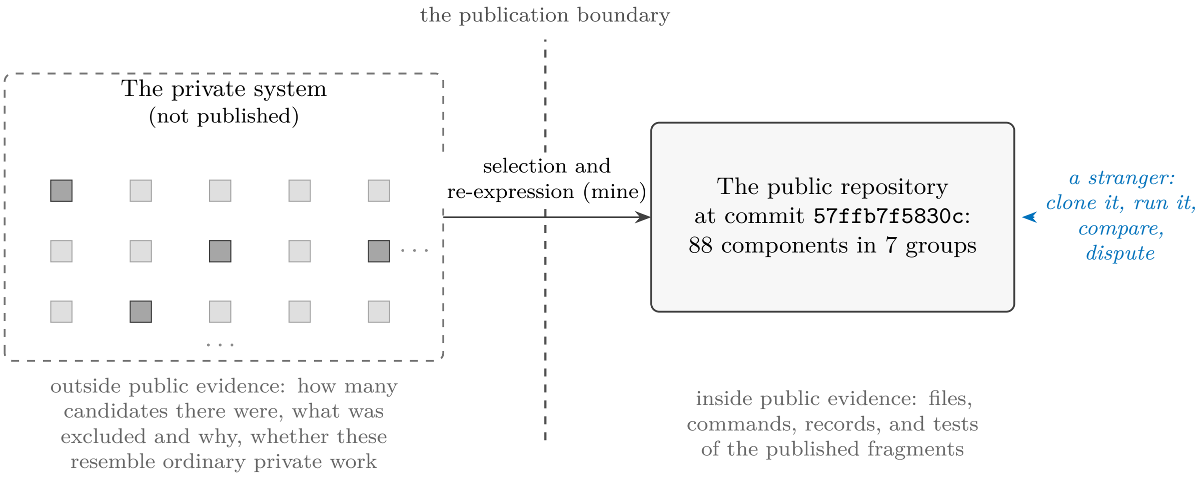

Figure 2: The selection problem. Each square stands for a private mechanism; the darker ones were given public form, and the grid trails off because its true extent is exactly what the public side cannot know. The count of squares on the left, the rule that picked the darker ones, and the resemblance between the two sides are all invisible from the right, so the collection is exact evidence about the published fragments and, by itself, evidence about nothing else.

ordinary work in the private system. A collection can consist entirely of genuine items and still give an unrepresentative picture of what it was drawn from; a display case does exactly this whenever the curator keeps the failures in a drawer. Figure 2 draws the situation: public checking reaches everything to the right of the boundary, and the pool the selection was drawn from stays on the left.

The registry's count should be read in that light. A registry of 88 entries defines the published population exactly, which is what a later evaluation would need in order to sample from it and report failure rates against a fixed denominator. The count is an inventory, not a score, and even the unit of counting is a choice I control, since one mechanism could have been registered as several components, or several as one.

Risk reduction versus guarantee

The publication process runs checks whose honest description is that they reduce specific risks. Automated scans search copied material for specified patterns of credentials, personal information, and private file paths, and record what was omitted or rejected; a scan establishes that the scanner did not find the patterns it was designed to find, and nothing stronger. Pages such as the component map are generated from the registry, and tests compare page and registry, which prevents the particular failure of prose drifting away from records; a generator and its records can still be wrong together. Plectis is also designed not to read the private repository at all, and that design is itself public and testable. None of this amounts to a guarantee that no secret, no identifying detail, and no inconsistency survives. Publication remains a judgement about acceptable risk, and I made it.

The provenance and risk-reduction distinctions pull against each other in a way worth noticing. Establishing provenance would require someone to examine private material, and the privacy constraint that motivates the whole design forbids exactly that examination in public. Some claims are therefore permanently in the reported category, short of a confidential review. The honest treatment is to label them, not to dress them up as public proof through bookkeeping

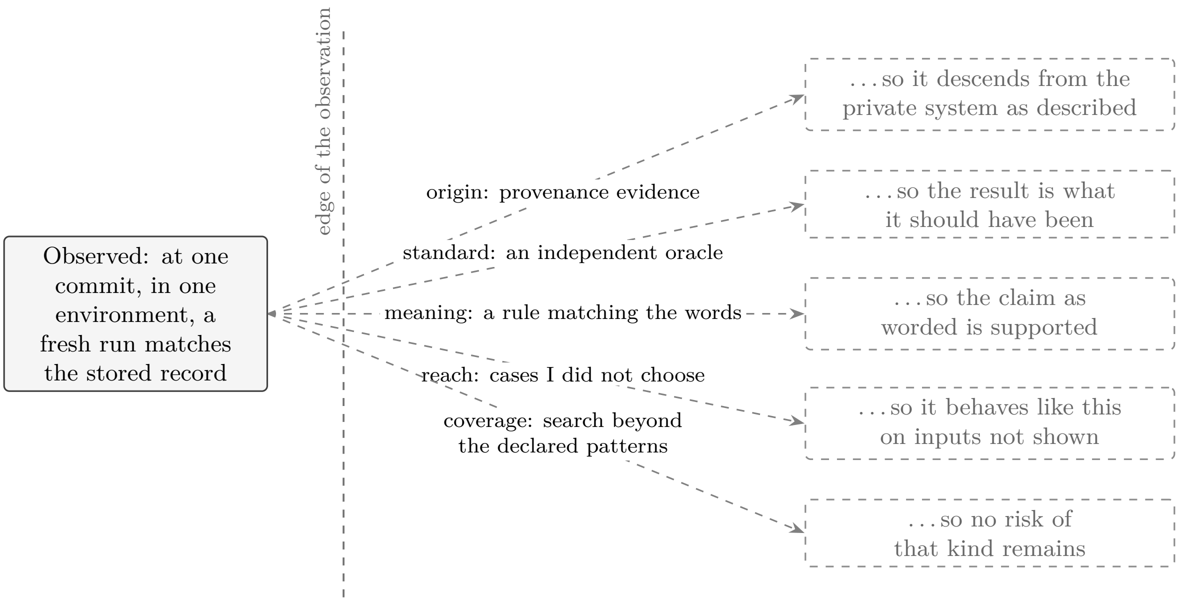

Figure 3: Five conclusions a matching run invites and does not license. Each dashed arrow needs a further premise that the observation itself cannot supply, and the tags follow the order of this section's distinctions: origin for provenance, standard for correctness, meaning for the validator's rule against the stated claim, reach for selection, and coverage for risk. The contract's contribution is not to cross these gaps but to make them nameable, one component at a time.

that one maintainer controls.

The five distinctions share one structure. What a stranger observes sits inside the published frame: a run completed, a record matched, chosen cases passed, a scan finished. The conclusion the observation invites lies outside the frame, and reaching it needs a premise the observation does not contain, one premise per distinction: that the public object descends from the private one; that the expectation is what the answer should have been; that the rule tests what the words say; that the chosen cases stand for the unchosen; that the risks searched for are the risks there are. The component contract does not supply these premises, and cannot. What it does is make each missing premise nameable, and a named gap can be argued about, priced, or deliberately left uncrossed, where an unnamed one is crossed by default.

5 What the collection contains

At commit `57ffb7f5830c` the 88 components fall into 7 groups:

Group	Count	What its components do
Getting started	2	Show a new reader where to begin and guide a first inspection.
Project mapping	12	Map files and written rules, find relevant material, and route a question to the part of the project that owns it.
Computer-checked mathematics	20	Prepare, repair, or check bounded steps used in formal mathematical work.
AI reliability and safety tests	20	Replay fixed cases involving hostile instructions, software boundaries, stored information, tool use, and benchmark rules.
Research-method tests	9	Exercise fixed cases involving replication, conflicting forecasts, explaining model behaviour, simulation, and laboratory safety.
Public-copy maintenance	20	Check copied source, generated pages, publication boundaries, and disagreement between records and derived files.
Work recovery	5	Record unfinished changes and test how work resumes after interruption.

The groups do not share a subject; computer-checked mathematics has nothing in particular to do with audio levels or with recovering interrupted work. Their unity is the contract of Section 2: these are the places in the private system where a mechanism could be given a bounded, runnable public form. The collection is organised by how its claims are checked, not by subject.

The registry also records how each public form was produced. Four routes cover all 88 entries, and each is a different trade between fidelity to the private original and what a stranger can check:

Route	Count	What it preserves	What it cannot show
Record-checked copied source	21	The exact text of non-secret source files, checked against a public import record.	That the recorded origin is real; record and file share one maintainer.
Public reimplementation	27	The behaviour of a bounded calculation, exercised on fixed cases.	Any relation to the private original beyond my report; the original text is absent.
Direct or external-tool run	22	A local computation, or a named external tool's result, captured with its context.	Behaviour where the tool or environment differs; the result is as strong as the tool and the case.
Contract checker	18	Whether a public input satisfies a declared structural rule.	Whether the declared rule is the right rule to require.

The copied-source route preserves the most text and demonstrates the least behaviour. A reimplementation demonstrates behaviour while surrendering textual identity; the worked example is one of the 27 in that route. An external-tool run borrows the tool's authority along with the tool's limits. A contract checker is worth exactly the contract its author chose to require. A reader who wants to know which trade a given component made can read its registry entry, which names its operation, command, receipt paths, evidence route, and limit.

Two cautions govern any reading of these tables. The first caution is arithmetic: components are not independent witnesses. Entries share code, fixture style, validator authorship, and my judgement; several entries can descend from one private mechanism, and where they do, their verdicts tend to move together. 88 entries are 88 registered claims, not 88 separate confirmations of anything. The same discount applies across the routes: the four routes are four different ceilings on inference, not four corroborating methods, and agreement among the pages, registries, and receipts that I generate from one source of truth is one witness, recorded several times. Machine-checked consistency among those records raises the cost of accident, not the number of witnesses.

The second caution is association. Group names such as computer-checked mathematics and AI reliability and safety tests borrow gravity from serious fields, and the borrowing happens in a reader whether or not any sentence asserts it. The corrective is to restate each group as the weakest property its validators actually check, which is often that a fixed case replays or that a record matches a declared shape, and then see how much of the impression survives. The names are for finding components, not for weighing them.

6 The author's hand

Every distinction in Section 4 ends at the same door, and it is better opened than pointed at. At the commit this paper describes, I hold every consequential choice on both sides of every test. I chose which private mechanisms were candidates, which were selected, and how finely each was divided into components, and so what the count counts. I chose which of the four routes each took, which inputs were frozen, what the expected values are, what each validator checks, and where each refusal boundary sits. And I chose which group each entry is filed under, which failures this paper explains and in what tone, which single component the worked example honours, and where the publication boundary lies. Somebody had to make each of these choices, and the privacy constraint meant it was me. None of that is misconduct.

But that concentration of choice has a known failure mode, and it involves no false statement anywhere. Select the mechanisms that show well. Divide the favourable ones finely. Narrow each claim until its fixed cases satisfy it. Route awkward material through the cheapest form of checking. Declare as refusals whatever would otherwise fail. File small replays under weighty names. Then let the counts, the categories, and the machinery of records accumulate into an impression of scale and maturity that no individual sentence asserts. A document can be cautious sentence by sentence while its arrangement still overstates, because a reader's impression forms on what is counted, named, worked through, and left out, as much as on what is claimed. This paper is inside its own arrangement, so declaring the risk does not discharge it; nothing in the text can establish that the text is not doing what this paragraph describes.

A quieter displacement deserves its own sentence. Components make disagreement small, and smallness cuts both ways: a reader can win an argument about one input or one rule while the question that brought them here, whether the selection is fair, whether the whole is sound, what any of it is for, goes unaddressed. An objection about selection is not answered by the successful execution of any number of selected items. The registry's job is to host the small arguments, not to stand in for the large ones.

Three defences need no cooperation from me. Read any component at the weakest property its validator checks, as Section 5 proposes. Treat every count as an inventory, and every agreement among my records as one witness. And weigh what the format leaves out: a mechanism survives selection when it can be made bounded, deterministic, and runnable in isolation, so a collection like this over-represents pure calculation, replay, and record-checking, and under-represents the stateful, entangled, long-running behaviour on which real software mostly stands or falls. The published picture tilts towards what publishes well before any question of honesty arises.

The fourth defence is structural rather than textual, and it is the subject of Section 7: evidence

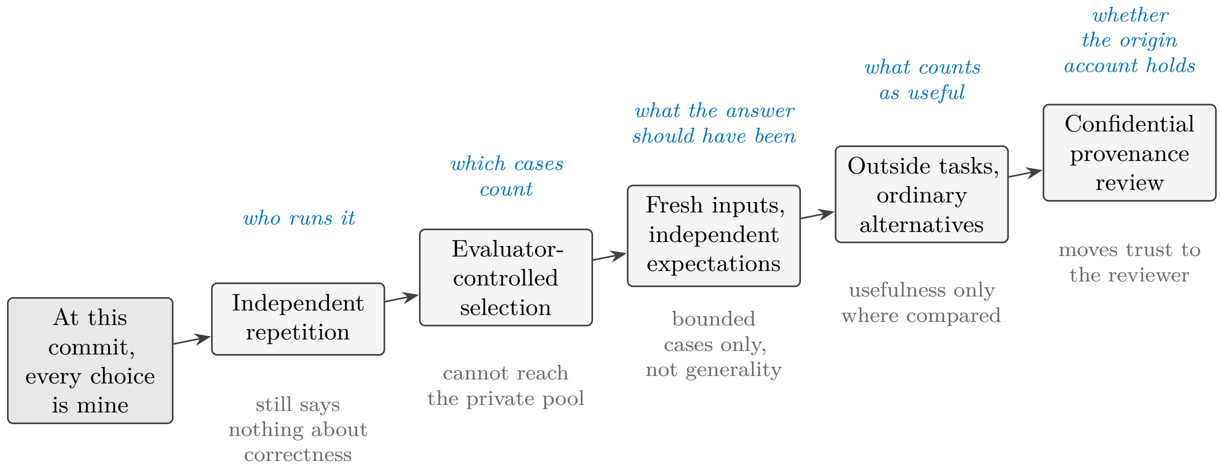

Figure 4: Stronger evidence as a transfer of control. Above each stage, in blue, the choice that leaves my hands; below it, in grey, what that stage still cannot show. At the commit this paper describes, none of the five stages has happened.

about an object like this becomes stronger as the consequential choices stop being mine.

7 What stronger evidence would look like

A passing run shows that named public code produced a stated result from a supplied input in a stated environment. Applying Section 4: the runs reported here do not establish private provenance, representativeness, correctness beyond self-supplied criteria, or anything about usefulness, security, originality, or the reliability of the private system as a whole. The computer-checked mathematics components prove no more than their named public fixtures, and no other repository's size or subject matter is used as evidence here. This note does not report an independent, randomly selected audit over fresh inputs; nothing of that kind has happened yet.

What would move each boundary is known, and the stages are ordered by who controls the consequential choices. Figure 4 shows the sequence; each stage transfers one class of choice out of my hands and leaves the later classes untouched.

Independent repetition comes first: a person with no involvement in the project reruns the supplied commands in their own environment and publishes what happened, including failures. That would remove the dependence on my machine and my tacit knowledge. It would say nothing about correctness.

Evaluator-controlled selection comes next: the evaluator chooses components from the full registry, without substitution when one proves awkward, and original failures stay on the record even if I later repair them. The units themselves are open to the same challenge: an evaluator who finds that two entries are one dependent mechanism, or that a boundary excludes the difficult part of a task, is disputing the registry, not misusing it. That addresses curation within the public collection. It cannot address whether the collection represents the private system, because the private pool stays invisible.

Fresh inputs with independently derived expectations move repeatability towards correctness: inputs chosen after a version is frozen, and expected results worked out through some route that does not pass through my code, whether independent calculation, a second implementation, or an external reference. Bounded correctness claims for the evaluated cases would then be available; general correctness still would not.

Comparison on outside tasks is what a usefulness claim would need: someone bringing a task that was not designed around the project's own categories, and comparing the component against an ordinary alternative, which for several of these components would be a short conventional script. Usefulness is comparative, and nothing in this paper compares.

A provenance review is the only stage that could bear on the origin story: a trusted person examining selected private material against the public collection under confidentiality. Even completed, this stage would transfer trust rather than remove it: public readers would be relying on the reviewer's access, competence, and report instead of on mine, and the review itself could not be published. The privacy cost may reasonably never be paid. If it is not, the origin story remains testimony, and this paper is written so that its argument does not depend on it.

How the record should age

The stages above will produce failures if they happen at all, and the value of the record then depends on what a failure does to it. The rule this project adopts is that repairs add facts and do not subtract them. The appendix records that at the pinned commit three public tests failed, and that a later commit repairs one of them; the repair is a dated fact about a later version, and it does not subtract the first fact about this one. The same rule holds above the level of code. If outside criticism shows a validator too weak or a claim too broad, the revision should be legible as a revision, carrying its date and its cause, rather than the project appearing always to have meant the narrower thing. A test rewritten after its outcome is known is a better test but a different one: passing it answers the earlier finding, and only cases fixed before their outcomes were known count as fresh confirmation. Disappointment is in scope: an evaluation that concludes a component is trivial, or no better than a short ordinary script, is the machinery working, and its conclusion belongs on the record with the same standing as a pass. An outside evaluator's report, where one comes to exist, should remain a separate document under the evaluator's control, cited rather than absorbed. Where the repository's future conduct falls short of this paragraph, the shortfall is itself a finding, and Section 8 says where findings go.

Each stage moves one class of consequential choice out of my hands. That, rather than more components or more records, is what stronger evidence means for an object like this.

8 What to inspect

The useful first review is small. Clone the repository, run the guided first inspection (`plectis tour`, shown with its exact commands in the appendix), and pick one claim you doubt, rather than the component most likely to impress. Read the component's description and stated limit before running anything. Run its command on the supplied input and compare the fresh output with the stored receipt. Then read the validator and ask whether its pass rule bears on the prose claim; perturb the input where practical; and keep whatever fails, because a discrepancy is a finding, and `CONTRIBUTING.md` in the repository sets out how to report one. One successful run supports one bounded conclusion. Broader conclusions wait on the evidence of Section 7, and most of it does not yet exist.

A A dated reproduction record

Everything in this appendix is an observation about one commit, made on 17 July 2026 in one environment: macOS 15.6.1 (arm64) with Python 3.13.7. The commit's full identifier is

```
57ffb7f5830cc446a2cbd8083d5d80bcab2af563
```

abbreviated `57ffb7f5830c` elsewhere in the paper. None of this appendix is a permanent property of the project. Later commits will change file counts, and other machines can behave differently; where that matters, the body of the paper does not depend on it.

Names. The repository is github.com/wcook04/plectis. As Section 2 notes, the code package inside it is named `microcosm_core`. The installed command is `plectis`, and an older command name, `microcosm`, still works as an alias.

Getting the code and running without installing.

```
git clone https://github.com/wcook04/plectis
cd plectis
PYTHONPATH=src python3 -m plectis tour --format text .
```

The final full stop in the tour command means "the current folder". To reproduce the exact state this appendix describes, run `git checkout 57ffb7f5830c` after `cd plectis`; without it the commands run against the newest version instead.

In a clean clone at this commit, the tour read 5,018 project files, found five possible reading orders for them, selected the one that starts from the README (`readme_onboarding_route`), wrote 21 files under a new `.microcosm/` directory beside the project, and reported the examined source files unchanged. Its output shows that every recorded observation carries three things: the evidence behind it, the event record it came from, and the limit on what it authorises. The `.microcosm/` directory can be deleted and generated again. The repository's README shows the same command reporting a different file count from a different day. Both numbers are what they claim to be, dated outputs of one run each, and neither is a stable measurement of the project.

Installing.

```
python3 -m pip install .
plectis tour --format text .
```

The installed package needs nothing beyond Python itself to run: it declares no third-party runtime dependencies. The build step requires a sufficiently recent `setuptools` (a standard Python packaging tool), which a machine building a Python package for the first time may download when `pip` builds this one. The core components are designed to make no network or AI-model calls when they run. No registered check enforces that absence: it is a design claim open to inspection, not a tested property, and a doubting reader should search the shipped source rather than trust this sentence. Before anything is installed, `./bootstrap.sh` checks a fresh clone and reports whether the shipped inputs and declared limits are intact.

The worked example. The command in Section 3 writes its fresh output under `/tmp/plectis-audio-rms`. The tracked reference to compare it against is the receipt named in Section 3, under `receipts/first_wave/batch8_audio_level_rms_port/`. At this commit the run reported every case in the Section 3 table as a pass, with observed values equal to the stored ones.

Project-wide checks. At this commit, `make check` loads the component registry and finishes in under a second, and `python3 scripts/check_lean_proof_trust.py` scans the 58 Lean proof files shipped in the repository for constructs that would let a file look proved without being proved: placeholder proofs, axioms the project granted itself, evaluation that bypasses the checker's core, declarations marked unsafe or partial, and raised limits on how much the checker examines. Both passed. (The 58 files are a different unit of counting from the 20 computer-checked mathematics components, so the two numbers are not comparable.) `make ci` adds installation, the public test suite, and two end-to-end example runs; it is the same floor the repository's continuous-integration service (the service that reruns the public checks after every change) runs. These commands test named properties of the public checkout. They do not certify the private system.

The full public test suite at this commit was not green: three of its tests failed, one because the repository's front-door profile check did not classify this paper's own files at the repository root, and two in documentation- and record-consistency checks that predate this revision. The profile defect is repaired in a later commit; under the rule of Section 7, that repair is a fact about the later commit and leaves this record standing. The failures are recorded here rather than smoothed over, which is what Section 8 asks a reader to do with their own failed runs.

The paper itself. A script, `scripts/check_public_system_paper.py`, recomputes every count used in this paper from the public registries, verifies that the named commit exists, verifies that the worked example and its receipt are the ones the registry declares, checks that the sentences carrying this paper's distinctions and cautions are still present, and fails if the paper and the repository disagree. The public test suite runs it, so drift between this document and the records it describes is a test failure. That makes the paper one of the repository's checked documents, and the check should be read with Section 4 in mind: agreement between a document and records with one maintainer is internal consistency, not external truth.